Análise de Algoritmos

Estruturas de Dados



UnB | CIC

Introdução

Quando dois programas resolvem o mesmo problema, mas parecem diferentes, será que um é melhor do que o outro?

Introdução

Quando dois programas resolvem o mesmo problema, mas parecem diferentes, será que um é melhor do que o outro?

- **Algoritmo** é uma lista passo a passo genérica de instruções para resolver um problema.

Introdução

Quando dois programas resolvem o mesmo problema, mas parecem diferentes, será que um é melhor do que o outro?

- **Algoritmo** é uma lista passo a passo genérica de instruções para resolver um problema.
- **Programa** é um algoritmo que foi codificado em alguma linguagem de programação. Pode haver diversos programas para o mesmo algoritmo, dependendo do programador e da linguagem de programação utilizada.

Introdução

- Problema: Calcular a soma dos *n* primeiros inteiros.

Introdução

- Problema: Calcular a soma dos *n* primeiros inteiros.
- Implementações diferentes de um algoritmo iterativo:

```
def somaDeN(n):
    aSoma = 0
    for i in range(1,n+1):
        aSoma = aSoma + i

    return aSoma
```

```
def foo(tom):
    fred = 0
    for bill in range(1,tom+1):
       barney = bill
       fred = fred + barney

    return fred
```

Introdução

- Problema: Calcular a soma dos *n* primeiros inteiros.
  - A função `somaDeN` é certamente melhor do que a função `foo` se você está preocupado com a legibilidade.
  - A função `foo` realiza uma atribuição a mais do que a função `somaDeN`
  - De acordo com a área da computação que estuda **Análise de Algoritmos**, as funções são igualmente ruins.

O que é Análise de Algoritmos?

*A análise de algoritmos se preocupa com a **comparação de algoritmos** baseada na quantidade de **recursos computacionais** que cada algoritmo usa.*

O que é Análise de Algoritmos?

*A análise de algoritmos se preocupa com a **comparação de algoritmos** baseada na quantidade de **recursos computacionais** que cada algoritmo usa.*

- Queremos comparar dois algoritmos e dizer quem é mais eficiente.
- Tipos de recursos computacionais:
    - Tempo de execução
    - Espaço ou memória
    - Largura de banda
    - … e etc

O que é Análise de Algoritmos?

Medindo tempo de execução da função soma de N números

```
import time

def somaDeN2(n):
    inicio = time.time()

    aSoma = 0
    for i in range(1, n+1):
        aSoma = aSoma + i

    fim = time.time()

    return aSoma, fim-inicio
```

O que é Análise de Algoritmos?

Executando a função soma de N números

```
>>> for i in range(5):
        print("A soma é %d demorou %10.7f segundos"%somaDeN2(10000))
A soma é 50005000 demorou  0.0018950 segundos
A soma é 50005000 demorou  0.0018620 segundos
A soma é 50005000 demorou  0.0019171 segundos
A soma é 50005000 demorou  0.0019162 segundos
A soma é 50005000 demorou  0.0019360 segundos
```

O que é Análise de Algoritmos?

Executando a função soma de N números

```
>>> for i in range(5):
        print("A soma é %d demorou %10.7f segundos"%somaDeN2(100000))
A soma é 5000050000 demorou  0.0199420 segundos
A soma é 5000050000 demorou  0.0180972 segundos
A soma é 5000050000 demorou  0.0194821 segundos
A soma é 5000050000 demorou  0.0178988 segundos
A soma é 5000050000 demorou  0.0188949 segundos
```

O que é Análise de Algoritmos?

Executando a função soma de N números

```
>>> for i in range(5):
        print("A soma é %d demorou %10.7f segundos"%somaDeN2(1000000))
A soma é 500000500000 demorou  0.1948988 segundos
A soma é 500000500000 demorou  0.1850290 segundos
A soma é 500000500000 demorou  0.1809771 segundos
A soma é 500000500000 demorou  0.1729250 segundos
A soma é 500000500000 demorou  0.1646299 segundos
```

O que é Análise de Algoritmos?

Problema: Calcular a soma dos *n* primeiros inteiros.

O que é Análise de Algoritmos?

Problema: Calcular a soma dos *n* primeiros inteiros.
Implementação em python usando a Função de Gauss:

```
def somaDeN3(n):
   return (n*(n+1))/2
```

O que é Análise de Algoritmos?

Problema: Calcular a soma dos *n* primeiros inteiros.
Implementação em python usando a Função de Gauss:

```
def somaDeN3(n):
   return (n*(n+1))/2
```

```
A soma é 50005000 demorou 0.00000095 segundos
A soma é 5000050000 demorou 0.00000191 segundos
A soma é 500000500000 demorou 0.00000095 segundos
A soma é 50000005000000 demorou 0.00000095 segundos
A soma é 5000000050000000 demorou 0.00000119 segundos
```

O que é Análise de Algoritmos?

- Os tempos de `somaDeN3` são mais curtos do que `somaDeN2`
- `somaDeN3` não é impactada pelo número de inteiros sendo somados.
- O tempo necessário para a solução iterativa parece aumentar conforme aumentamos o valor de *n*
- Será que esse resultado se mantém em máquinas, sistemas operacionais, linguagens, interpretadores, compiladores e dias diferentes?








O que é Análise de Algoritmos?

- Os tempos de `somaDeN3` são mais curtos do que `somaDeN2`
- `somaDeN3` não é impactada pelo número de inteiros sendo somados.
- O tempo necessário para a solução iterativa parece aumentar conforme aumentamos o valor de *n*
- Será que esse resultado se mantém em máquinas, sistemas operacionais, linguagens, interpretadores, compiladores e dias diferentes?

Gostaríamos de ter uma caracterização independente do programa ou computador sendo utilizados

Notação O

- Tempo não é uma boa medida
- O número de passos necessários é mais interessante
- Definir os **passos** ou **unidade básica** de computação depende do tipo de algoritmo. A unidade básica de computação para ED é o número de operações (instruções de atribuição)

Notação O

- Tempo não é uma boa medida
- O número de passos necessários é mais interessante
- Definir os **passos** ou **unidade básica** de computação depende do tipo de algoritmo. A unidade básica de computação para ED é o número de operações (instruções de atribuição)

$$T(n) = n + 1$$

"T(n) é o tempo gasto ou consumido para resolver um problema de tamanho n, ou seja, 1 + n passos."

Notação O

- O número exato de operações T(n) não é tão importante
- Preferimos olhar os termos dominantes!
- Então o que importa é a **ordem de magnitude**
- Ordem de magnitude é frequentemente chamado de **notação O grande**

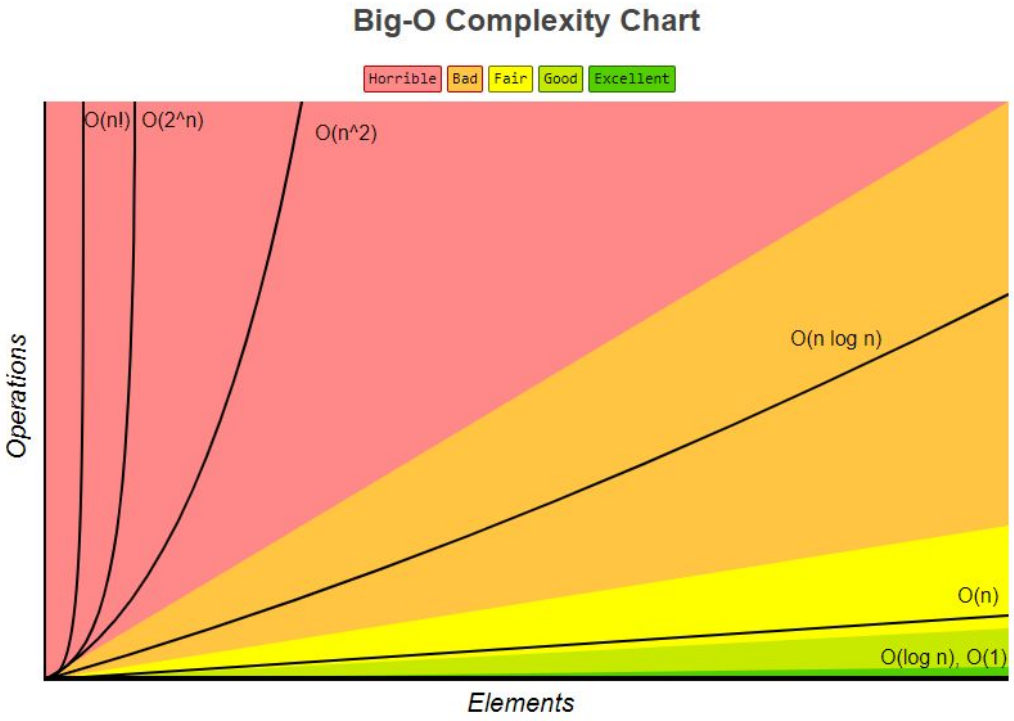
Notação O

- O número exato de operações T(n) não é tão importante
- Preferimos olhar os termos dominantes!
- Então o que importa é a **ordem de magnitude**
- Ordem de magnitude é frequentemente chamado de **notação O grande**

$$T(n) = n + 1 \text{ tem ordem de magnitude } n \sim O(n)$$

$$T(n) = 5n^2 + 27n + 1005 \text{ tem ordem de magnitude } n^2 \sim O(n^2)$$

Notação O


Big-O Complexity Chart
Horrible
Bad
Fair
Good
Excellent
O(n!)
O(2^n)
O(n^2)
O(n log n)
O(n)
O(log n), O(1)
Operations
Elements

Detecção de Anagramas

- Uma string é um anagrama de outra se a segunda é simplesmente um rearranjo da primeira.
- Por exemplo, 'amor' e 'roma' são anagramas.
- Por exemplo, 'marrocos' e 'socorram' também são anagramas.
- Nosso objetivo é escrever uma função booleana que receberá duas strings e devolverá se elas são anagramas.

Detecção de Anagramas (1)

```
def solucaoAnagrama1(s1,s2):
    umalista = list(s2)

    pos1 = 0
    aindaOK = True

    while pos1 < len(s1) and aindaOK:
        pos2 = 0
        encontrado = False
        while pos2 < len(umalista) and not encontrado:
            if s1[pos1] == umalista[pos2]:
                encontrado = True
            else:
                pos2 = pos2 + 1

        if encontrado:
            umalista[pos2] = None
        else:
            aindaOK = False

        pos1 = pos1 + 1

    return aindaOK
```

Detecção de Anagramas (1)

```
def solucaoAnagrama1(s1,s2):
    umalista = list(s2)

    pos1 = 0
    aindaOK = True

    while pos1 < len(s1) and aindaOK:
        pos2 = 0
        encontrado = False
        while pos2 < len(umalista) and not encontrado:
            if s1[pos1] == umalista[pos2]:
                encontrado = True
            else:
                pos2 = pos2 + 1

        if encontrado:
            umalista[pos2] = None
        else:
            aindaOK = False

        pos1 = pos1 + 1

    return aindaOK
```

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$T(n) = ½n^2 + ½n$

Detecção de Anagramas (1)

```
def solucaoAnagrama1(s1,s2):
    umalista = list(s2)

    pos1 = 0
    aindaOK = True

    while pos1 < len(s1) and aindaOK:
        pos2 = 0
        encontrado = False
        while pos2 < len(umalista) and not encontrado:
            if s1[pos1] == umalista[pos2]:
                encontrado = True
            else:
                pos2 = pos2 + 1

        if encontrado:
            umalista[pos2] = None
        else:
            aindaOK = False

        pos1 = pos1 + 1

    return aindaOK
```

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$T(n) = ½n^2 + ½n$

$O(n^2)$

Detecção de Anagramas (2)

```
def solucaoAnagrama2(s1,s2):
   umalista1 = list(s1)
   umalista2 = list(s2)

   umalista1.sort()
   umalista2.sort()

   pos = 0
   iguais = True

   while pos < len(s1) and iguais:
       if umalista1[pos]==umalista2[pos]:
           pos = pos + 1
       else:
           iguais = False

   return iguais
```

Detecção de Anagramas (2)

```
def solucaoAnagrama2(s1,s2):
   umalista1 = list(s1)
   umalista2 = list(s2)

   umalista1.sort()
   umalista2.sort()

   pos = 0
   iguais = True

   while pos < len(s1) and iguais:
       if umalista1[pos]==umalista2[pos]:
           pos = pos + 1
       else:
           iguais = False

   return iguais

print(solucaoAnagrama2('abcde','edcba'))
```

$O(n^2)$ ou $O(n \log n)$
Cuidado com a função sort.

Detecção de Anagramas (3)

```
def solucaoAnagrama3(s1,s2):
    c1 = [0]*26
    c2 = [0]*26

    for i in range(len(s1)):
        pos = ord(s1[i])-ord('a')
        c1[pos] = c1[pos] + 1

    for i in range(len(s2)):
        pos = ord(s2[i])-ord('a')
        c2[pos] = c2[pos] + 1

    j = 0
    aindaOK = True
    while j<26 and aindaOK:
        if c1[j]==c2[j]:
            j = j + 1
        else:
            aindaOK = False

    return aindaOK
```

Detecção de Anagramas (3)

```
def solucaoAnagrama3(s1,s2):
    c1 = [0]*26
    c2 = [0]*26

    for i in range(len(s1)):
        pos = ord(s1[i])-ord('a')
        c1[pos] = c1[pos] + 1

    for i in range(len(s2)):
        pos = ord(s2[i])-ord('a')
        c2[pos] = c2[pos] + 1

    j = 0
    aindaOK = True
    while j<26 and aindaOK:
        if c1[j]==c2[j]:
            j = j + 1
        else:
            aindaOK = False

    return aindaOK
```

T(n) = 2n + 26
O(n)

Exercícios

Qual a complexidade assintótica desses blocos de códigos?

```
teste = 0
for i in range(n):
   for j in range(n):
       teste = teste + i * j
```

```
teste = 0
for i in range(n):
   teste = teste + 1
for j in range(n):
    teste = teste - 1
```

```
i = n
while i > 0:
   k = 2 + 2
   i = i // 2
```

Complexidade das Estruturas de Dados em Python

- As operaçoẽs em lista e dicionários tem uma complexidade computacional.
- Algumas operações têm complexidade maior e outras menores.
- Essa complexidade depende das escolhas realizadas pela linguagem no momento da implementação dessas estruturas.
- Exemplo:

```
teste = []
for i in range(n):
    teste.insert(0, input())
```

```
teste = []
for i in range(n):
    teste.append(input())
```

Complexidade da Lista

Operação	Complexidade Assintótica
Indexação []	O(1)
Atribuição	O(1)
append	O(1)
pop	O(1)
pop(i)	O(n)
insert(i, item)	O(n)
fatiar [x:y]	O(k)
contém (in)	O(n)
ordenar	O(n log n)

Complexidade do Dicionário

Operação	Complexidade Assintótica
acessar item	O(1)
atribuir item	O(1)
remover item	O(1)
contém (in)	O(1)

Resumo

- A análise de algoritmos é uma maneira de medir um algoritmo independentemente da implementação.
- A notação O permite que algoritmos sejam classificados por seu processo dominante em relação ao tamanho do problema.
- As estruturas de python tem diferentes complexidades assintóticas.